

006 — VPS Setup Guide

 A VPS (Virtual Private Server) is a remote Linux machine in the cloud where we deploy our application. We'll use DigitalOcean to create a "Droplet" (their name for a VPS) and configure it for Docker deployment.

Table of Contents

- 1. [Creating a Droplet](#)
 - 2. [First SSH Login](#)
 - 3. [Initial Server Security](#)
 - 4. [Installing Docker](#)
 - 5. [UFW Firewall Configuration](#)
 - 6. [Directory Structure for Deployment](#)
 - 7. [Creating a Managed Database](#)
 - 8. [Verifying Everything Works](#)
 - 9. [Useful Server Commands](#)
 - 10. [Quick Reference Card](#)
-

1. Creating a Droplet

1.1 Navigate to Droplet Creation

- 1. Log in to <https://cloud.digitalocean.com>
- 2. Click **Create** (green button, top-right) → **Droplets**

1.2 Choose Configuration

Setting	Recommended Choice
Region	Choose closest to you (e.g., Singapore, Frankfurt)
Image	Ubuntu 24.04 (LTS) x64
Droplet Type	Basic (Shared CPU)
CPU Options	Regular (SSD)
Size	\$6/mo — 1 GB RAM / 1 CPU / 25 GB SSD / 1000 GB Transfer
Authentication	SSH Key (select the key you added earlier)
Hostname	Something descriptive: <code>kanban-prod</code> or <code>bootcamp-yourname</code>

 **Why \$6/mo (1GB RAM)?**

- Sufficient for running 4-5 Docker containers (Caddy + API + Web + Admin + Watchtower)
- If you experience memory issues, you can resize later

- For production with more traffic, consider \$12/mo (2GB RAM)

1.3 Select Authentication Method

Option A: SSH Key (Recommended)

1. Click **New SSH Key**
2. On your local machine, copy your public key:

```
cat ~/.ssh/id_ed25519.pub
```

3. Paste it into the form
4. Name it (e.g., **My Laptop**)
5. Click **Add SSH Key**

Option B: Password (Fallback)

If SSH key setup failed, select "Password" and create a strong root password. You'll change this later.

1.4 Additional Options (Optional)

-  **Monitoring** — Free, adds CPU/RAM/disk graphs to dashboard
- ☐ IPv6 — Not needed for bootcamp
- ☐ User data — Not needed

1.5 Create the Droplet

1. Click **Create Droplet**
2. Wait 30-60 seconds for creation
3. **Copy the IP address** that appears (e.g., **167.71.123.45**)

 **Write down your Droplet IP!** You'll use it throughout the bootcamp. Send it to the instructor for DNS configuration.

1.6 One-Click Docker Alternative

DigitalOcean offers a **Docker pre-installed image**:

1. During droplet creation, go to **Marketplace** tab
2. Search for **Docker**
3. Select **Docker on Ubuntu 24.04**

This skips [Section 4 \(Installing Docker\)](#) entirely.

2. First SSH Login

2.1 Connect to Your Droplet

From your local terminal (WSL Ubuntu or macOS Terminal):

```
ssh root@YOUR_DROPLET_IP
```

Replace **YOUR_DROPLET_IP** with the IP from step 1.5.

First-time connection prompt:

```
The authenticity of host '167.71.123.45' can't be established.  
ED25519 key fingerprint is SHA256:abc123def456...  
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Type **yes** and press Enter.

2.2 Verify You're Connected

You should see something like:

```
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.x.x-x-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
root@kanban-prod:~#
```

You're now on the remote server! 🎉

2.3 Add SSH Config Entry (Local Machine)

Back on your **local machine**, add a shortcut:

```
# Add to ~/.ssh/config  
cat >> ~/.ssh/config << 'EOF'  
  
Host vps  
  HostName YOUR_DROPLET_IP  
  User root  
  IdentityFile ~/.ssh/id_ed25519  
  ServerAliveInterval 60  
  ServerAliveCountMax 3  
EOF
```

Replace **YOUR_DROPLET_IP** with your actual IP.

Now you can connect with just:

```
ssh vps
```

2.4 If SSH Fails

Error	Cause	Fix
Connection refused	Droplet not ready yet	Wait 1 minute, try again
Connection timed out	Wrong IP or firewall	Verify IP on DO dashboard
Permission denied	Key mismatch	See SSH Cheatsheet, Section 6
Host key verification failed	IP was reused	<code>ssh-keygen -R YOUR_IP</code>

3. Initial Server Security

Run these commands **on the VPS** (after SSH-ing in).

3.1 Update System Packages

```
apt update && apt upgrade -y
```

3.2 Set Timezone

```
timedatectl set-timezone Asia/Kuala_Lumpur
```

Verify:

```
timedatectl  
# Should show your timezone
```

💡 Find your timezone: `timedatectl list-timezones | grep Asia`

3.3 Install Essential Tools

```
apt install -y \  
  curl \  
  wget \  
  git \  
  nano \  
  htop \  
  ncd� \  
  unzip \  
  \
```

```
net-tools \
dnsutils \
ca-certificates \
gnupg \
lsb-release
```

Tool	Purpose
<code>curl</code> / <code>wget</code>	Download files
<code>git</code>	Version control (if needed on server)
<code>nano</code>	Simple text editor
<code>htop</code>	System resource monitor (better than <code>top</code>)
<code>ncdu</code>	Disk usage analyzer
<code>net-tools</code>	Network utilities (<code>netstat</code> , etc.)
<code>dnsutils</code>	DNS tools (<code>dig</code> , <code>nslookup</code>)

3.4 (Optional) Create a Non-Root User

For production servers, running as root is discouraged. For a bootcamp, root is fine.

If you want to set it up properly:

```
# Create user
adduser deploy

# Give sudo access
usermod -aG sudo deploy

# Copy SSH key to new user
rsync --archive --chown=deploy:deploy ~/.ssh /home/deploy

# Test login (from local machine)
ssh deploy@YOUR_DROPLET_IP
```

4. Installing Docker



Skip this section if you used the DigitalOcean Docker Marketplace image.

4.1 Install Docker Engine

Run these commands on the VPS:

```
# Remove any old Docker installations
apt remove -y docker docker-engine docker.io containerd runc 2>/dev/null
```

```
# Add Docker's official GPG key
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg

# Add Docker repository
echo \
  "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine + Compose Plugin
apt update
apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

4.2 Verify Docker Installation

```
# Check Docker version
docker --version
# Docker version 27.x.x, build xxxxx

# Check Docker Compose version
docker compose version
# Docker Compose version v2.x.x

# Run test container
docker run --rm hello-world
```

You should see "Hello from Docker!" 🎉

4.3 Enable Docker to Start on Boot

```
systemctl enable docker
systemctl enable containerd
```

4.4 Verify Docker is Running

```
systemctl status docker
```

Should show **active (running)**.

5. UFW Firewall Configuration

🔥 UFW (Uncomplicated Firewall) controls which ports are accessible from the internet. We only want to expose SSH (22), HTTP (80), and HTTPS (443).

5.1 Check Current Status

```
ufw status
# Status: inactive (initially)
```

5.2 Set Default Policies

```
# Deny all incoming traffic by default
ufw default deny incoming

# Allow all outgoing traffic
ufw default allow outgoing
```

5.3 Allow Required Ports

```
# SSH (CRITICAL – don't lock yourself out!)
ufw allow 22/tcp comment "SSH"

# HTTP (for Caddy / Let's Encrypt)
ufw allow 80/tcp comment "HTTP"

# HTTPS (for production traffic)
ufw allow 443/tcp comment "HTTPS"
```

5.4 Enable the Firewall

```
ufw enable
```

You'll see:

```
Command may disrupt existing SSH connections. Proceed with operation (y|n)?
y
Firewall is active and enabled on system startup
```

⚠️ **Make sure you allowed port 22 BEFORE enabling!** Otherwise you'll lock yourself out.

5.5 Verify Firewall Rules

```
ufw status verbose
```

Expected output:

```
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)

To Action From
--
22/tcp ALLOW IN Anywhere # SSH
80/tcp ALLOW IN Anywhere # HTTP
443/tcp ALLOW IN Anywhere # HTTPS
22/tcp (v6) ALLOW IN Anywhere (v6) # SSH
80/tcp (v6) ALLOW IN Anywhere (v6) # HTTP
443/tcp (v6) ALLOW IN Anywhere (v6) # HTTPS
```

5.6 UFW Quick Commands

Command	Description
<code>ufw status</code>	Show current rules
<code>ufw status numbered</code>	Show rules with numbers
<code>ufw allow 8080/tcp</code>	Open a port
<code>ufw deny 8080/tcp</code>	Block a port
<code>ufw delete 3</code>	Delete rule #3 (use <code>status numbered</code> first)
<code>ufw delete allow 8080/tcp</code>	Delete by rule
<code>ufw disable</code>	Turn off firewall
<code>ufw reset</code>	Remove all rules
<code>ufw reload</code>	Reload rules

5.7 Important Note About Docker & UFW

⚠ Docker bypasses UFW by default!

When you use `-p 8080:80` in Docker, it opens port 8080 directly via iptables, completely bypassing UFW rules.

For our bootcamp setup, this is fine because:

- Only Caddy (gateway) exposes ports 80/443
- Other containers (api, web, admin) don't publish ports to the host
- They communicate through Docker's internal network

If you want to fix this for production, add to `/etc/docker/daemon.json`:

```
{
  "iptables": false
}
```

Then restart Docker: `systemctl restart docker`

But for the bootcamp, **don't do this** — leave the default behavior.

6. Directory Structure for Deployment

6.1 Create Project Directory

```
mkdir -p ~/kanban-prod
cd ~/kanban-prod
```

6.2 Create Required Files

We need three files in this directory:

```
~/kanban-prod/
├── .env           ← Secrets & configuration
├── Caddyfile      ← Reverse proxy configuration
└── docker-compose.yml ← Container orchestration
```

6.3 Create the `.env` File

```
nano ~/kanban-prod/.env
```

Paste the following (replace placeholder values):

```
# — Database (DigitalOcean Managed PostgreSQL) —
DATABASE_URL=Host=YOUR_DB_HOST;Port=25060;Database=defaultdb;Username=doadm
in;Password=YOUR_DB_PASSWORD;SSL Mode=Require;Trust Server Certificate=true

# — Admin Seed —
ADMIN_EMAIL=admin@kanban.local
ADMIN_PASSWORD=Admin123!
ADMIN_DISPLAY_NAME=System Admin

# — ASP.NET —
```

```
ASPNETCORE_ENVIRONMENT=Production
```

```
# — CORS & Cookies —————  
CORS_ORIGINS=https://YOUR_SUBDOMAIN.lazuar.com  
COOKIE_DOMAIN=YOUR_SUBDOMAIN.lazuar.com  
  
# — Watchtower (GHCR credentials for auto-updates) ————  
REPO_USER=YOUR_GITHUB_USERNAME  
REPO_PASS=ghp_YOUR_PERSONAL_ACCESS_TOKEN
```

Save: **Ctrl+X**, then **Y**, then **Enter**.

6.4 Create the Caddyfile

```
nano ~/kanban-prod/Caddyfile
```

Paste (replace domain):

```
YOUR_SUBDOMAIN.lazuar.com {  
  # API Traffic → .NET API container  
  handle /api/* {  
    reverse_proxy api:5010  
  }  
  
  # SignalR Hub (WebSocket support)  
  handle /api/hubs/* {  
    reverse_proxy api:5010  
  }  
  
  # Admin Panel → Admin Caddy container  
  handle /admin/* {  
    reverse_proxy admin:80  
  }  
  
  # Everything else → Next.js Web App  
  handle /* {  
    reverse_proxy web:3000  
  }  
}
```

Save: **Ctrl+X**, then **Y**, then **Enter**.

6.5 Create the docker-compose.yml

```
nano ~/kanban-prod/docker-compose.yml
```

Paste (replace GitHub org/repo paths if using your own fork):

```
services:
# — GATEWAY (Caddy – HTTPS + Reverse Proxy) —
gateway:
  image: caddy:2-alpine
  container_name: kanban-gateway
  restart: always
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./Caddyfile:/etc/caddy/Caddyfile
    - caddy_data:/data
    - caddy_config:/config
  depends_on:
    - api
    - web
    - admin

# — API (.NET) —
api:
  image: ghcr.io/proxeon/bootcamp/api:latest
  container_name: kanban-api
  restart: always
  environment:
    - DATABASE_URL=${DATABASE_URL}
    - ASPNETCORE_ENVIRONMENT=${ASPNETCORE_ENVIRONMENT}
    - ADMIN_EMAIL=${ADMIN_EMAIL}
    - ADMIN_PASSWORD=${ADMIN_PASSWORD}
    - ADMIN_DISPLAY_NAME=${ADMIN_DISPLAY_NAME}
    - CORS_ORIGINS=${CORS_ORIGINS}
    - COOKIE_DOMAIN=${COOKIE_DOMAIN}

# — WEB (Next.js) —
web:
  image: ghcr.io/proxeon/bootcamp/web:latest
  container_name: kanban-web
  restart: always

# — ADMIN (Vite + Caddy) —
admin:
  image: ghcr.io/proxeon/bootcamp/admin:latest
  container_name: kanban-admin
  restart: always

# — WATCHTOWER (Auto-Update from GHCR) —
watchtower:
  image: containrrr/watchtower
  container_name: kanban-watchtower
  restart: always
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
```

```
environment:
  - REPO_USER=${REPO_USER}
  - REPO_PASS=${REPO_PASS}
command: --interval 300 --cleanup

volumes:
  caddy_data:
  caddy_config:
```

Save: **Ctrl+X**, then **Y**, then **Enter**.

6.6 Login to GitHub Container Registry

Before Docker can pull images, authenticate with GHCR:

```
export GH_USERNAME="YOUR_GITHUB_USERNAME"
export CR_PAT="ghp_YOUR_TOKEN"

echo $CR_PAT | docker login ghcr.io -u $GH_USERNAME --password-stdin
```

Expected output:

```
Login Succeeded
```

6.7 Final Directory Verification

```
ls -la ~/kanban-prod/
```

Should show:

```
total 16
drwxr-xr-x 2 root root 4096 ... .
drwx----- 5 root root 4096 ... ..
-rw-r--r-- 1 root root 450 ... .env
-rw-r--r-- 1 root root 320 ... Caddyfile
-rw-r--r-- 1 root root 980 ... docker-compose.yml
```

7. Creating a Managed Database

 We use DigitalOcean Managed PostgreSQL so we don't have to manage database backups, updates, or failover ourselves.

7.1 Create the Database Cluster

1. Go to: <https://cloud.digitalocean.com/databases>
2. Click **Create Database Cluster**
3. Configure:

Setting	Value
Engine	PostgreSQL 17
Region	Same as your Droplet
Plan	Basic — \$15/mo (1 GB RAM, 1 vCPU, 10 GB Storage)
Cluster name	kanban-db

4. Click **Create Database Cluster**
5. Wait 3-5 minutes for provisioning

7.2 Secure the Database

After creation, go to the database **Settings** tab:

1. **Trusted Sources:** Click **Edit**
2. Add your **Droplet** (select it from the list or enter the IP)
3. Click **Save**

⚠ This is critical! Without adding your Droplet as a trusted source, the API container cannot reach the database.

7.3 Get the Connection String

1. Go to your database cluster's **Overview** tab
2. In the **Connection Details** section:
 - Select **Connection string** dropdown
 - Select **Format:** `Connection parameters`

You'll see:

```
host      = db-postgresql-sgp1-12345-do-user-xxxxx-0.c.db.ondigitalocean.com
port      = 25060
username  = doadmin
password  = AVNS_XXXXXXXXXXXXX
database  = defaultdb
sslmode   = require
```

7.4 Format for .NET Connection String

Convert the above into the format our API expects:

```
Host=db-postgresql-sgp1-12345-do-user-xxxxx-  
0.c.db.ondigitalocean.com;Port=25060;Database=defaultdb;Username=doadmin;Pa  
ssword=AVNS_XXXXXXXXXXXX;SSL Mode=Require;Trust Server Certificate=true
```

7.5 Update Your .env File

SSH into your VPS and update the `.env`:

```
ssh vps  
nano ~/kanban-prod/.env
```

Replace the `DATABASE_URL` line with your actual connection string.

7.6 Test Database Connectivity (From VPS)

```
# Install PostgreSQL client  
apt install -y postgresql-client  
  
# Test connection  
psql "host=YOUR_DB_HOST port=25060 dbname=defaultdb user=doadmin  
password=YOUR_PASSWORD sslmode=require"
```

If you see the `defaultdb=>` prompt, the connection works! Type `\q` to exit.

7.7 Alternative: Use Containerized PostgreSQL

If you want to skip the managed database (saves \$15/month):

Replace the `docker-compose.yml` to include PostgreSQL:

```
services:  
  # — DATABASE —————  
  postgres:  
    image: postgres:17  
    container_name: kanban-postgres  
    restart: always  
    environment:  
      POSTGRES_USER: kanban  
      POSTGRES_PASSWORD: kanban_secure_password_here  
      POSTGRES_DB: kanban  
    volumes:  
      - postgres_data:/var/lib/postgresql/data  
    healthcheck:  
      test: ["CMD-SHELL", "pg_isready -U kanban"]  
      interval: 5s  
      timeout: 5s
```

```

    retries: 5

# ... (rest of services stay the same, but api depends_on postgres)
api:
  # ... existing config ...
  depends_on:
    postgres:
      condition: service_healthy

volumes:
  postgres_data:
  caddy_data:
  caddy_config:

```

And update `.env`:

```

DATABASE_URL=Host=postgres;Port=5432;Database=kanban;Username=kanban;Password=kanban_secure_password_here

```

⚠ **With containerized DB:** If you run `docker compose down -v`, you **lose all data**. Managed databases don't have this risk.

8. Verifying Everything Works

8.1 Launch the Stack

```

cd ~/kanban-prod
docker compose up -d

```

8.2 Check Container Status

```

docker compose ps

```

Expected output (all should be "Up"):

NAME	IMAGE	STATUS
kanban-gateway	caddy:2-alpine	Up
0.0.0.0:80->80/tcp,	0.0.0.0:443->443/tcp	
kanban-api	ghcr.io/proxeon/bootcamp/api:latest	Up
kanban-web	ghcr.io/proxeon/bootcamp/web:latest	Up
kanban-admin	ghcr.io/proxeon/bootcamp/admin:latest	Up
kanban-watchtower	containrrr/watchtower	Up

8.3 Check Logs for Errors

```
# All services
docker compose logs --tail 20

# Specific service (check API for database connection)
docker compose logs --tail 30 api

# Check Caddy for SSL certificate
docker compose logs gateway | grep -i "certificate"
```

Look for these success indicators:

API logs:

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://[::]:5010
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand successfully
```

Gateway (Caddy) logs:

```
"msg":"certificate obtained successfully"
"msg":"enabling automatic TLS"
```

8.4 Test from Your Browser

Visit: https://YOUR_SUBDOMAIN.lazuar.com

You should see the Kanban application! 🎉

URL	What you should see
https://YOUR_SUBDOMAIN.lazuar.com	Web app (login page)
https://YOUR_SUBDOMAIN.lazuar.com/admin/	Admin panel
https://YOUR_SUBDOMAIN.lazuar.com/api/	"Kanban API is running!"

8.5 Test from Command Line (On VPS)

```
# Test API health
curl -s http://localhost:80/api/health
# Should return: Healthy

# Test with HTTPS (from anywhere)
```

```
curl -s https://YOUR_SUBDOMAIN.lazuar.com/api/  
# Should return: Kanban API is running!
```

8.6 If Something Is Wrong

```
# Full diagnostic  
echo "=== CONTAINER STATUS ==="  
docker compose ps  
  
echo ""  
echo "=== API LOGS (last 20) ==="  
docker compose logs --tail 20 api  
  
echo ""  
echo "=== GATEWAY LOGS (last 10) ==="  
docker compose logs --tail 10 gateway  
  
echo ""  
echo "=== DISK SPACE ==="  
df -h /  
  
echo ""  
echo "=== MEMORY ==="  
free -h  
  
echo ""  
echo "=== PORTS IN USE ==="  
ss -tlnp | grep -E ':(80|443|5010|3000)\s'
```

Common fixes:

```
# Container won't start – check logs  
docker compose logs api  
  
# Port already in use  
docker compose down && docker compose up -d  
  
# Out of memory  
docker system prune -af  
docker compose up -d  
  
# SSL not working – ensure DNS is pointing to this IP  
dig YOUR_SUBDOMAIN.lazuar.com  
  
# Rebuild everything from scratch  
docker compose down -v  
docker compose pull  
docker compose up -d
```

9. Useful Server Commands

9.1 System Monitoring

Command	Description
<code>htop</code>	Interactive process viewer (press <code>q</code> to exit)
<code>free -h</code>	Memory usage
<code>df -h</code>	Disk space
<code>du -sh /var/lib/docker</code>	Docker disk usage
<code>ncdu /</code>	Interactive disk usage explorer
<code>uptime</code>	Server uptime and load average
<code>w</code>	Who is logged in
<code>last</code>	Login history

9.2 Log Management

Command	Description
<code>journalctl -u docker --since "1 hour ago"</code>	Docker daemon logs
<code>journalctl -f</code>	Follow system logs
<code>tail -f /var/log/syslog</code>	System log
<code>tail -f /var/log/auth.log</code>	Authentication log (SSH attempts)

9.3 Network Diagnostics

Command	Description
<code>curl -I https://your-domain.com</code>	Check HTTP headers
<code>dig your-domain.com</code>	DNS lookup
<code>ss -tlnp</code>	Show listening ports
<code>netstat -tulpn</code>	Show all connections
<code>ping google.com</code>	Test internet connectivity
<code>traceroute google.com</code>	Trace network path

9.4 File Operations

Command	Description
<code>nano filename</code>	Edit file (save: Ctrl+X, Y, Enter)

Command	Description
<code>cat filename</code>	Display file contents
<code>less filename</code>	Scrollable file viewer
<code>head -20 filename</code>	First 20 lines
<code>tail -20 filename</code>	Last 20 lines
<code>find / -name "*.log" -size +100M</code>	Find large log files

9.5 Service Management

Command	Description
<code>systemctl status docker</code>	Docker service status
<code>systemctl restart docker</code>	Restart Docker
<code>systemctl status ufw</code>	Firewall status
<code>reboot</code>	Restart the server
<code>shutdown -h now</code>	Power off (careful!)

9.6 Quick Server Health Check Script

Create this script on your VPS:

```
cat > ~/check-health.sh << 'EOF'
#!/bin/bash
echo ""
echo ""
echo ""
echo ""

echo "— System —————"
printf "  Uptime:      %s\n" "$(uptime -p)"
printf "  Load:       %s\n" "$(cat /proc/loadavg | cut -d' ' -f1-3)"
printf "  Memory:      %s / %s\n" "$(free -h | awk '/Mem:/{print $3}')"
                    "$(free -h | awk '/Mem:/{print $2}')"
printf "  Disk:        %s / %s (%s used)\n" "$(df -h / | awk 'NR==2{print $3}')"
                    "$(df -h / | awk 'NR==2{print $2}')"
                    "$(df -h / | awk 'NR==2{print $5}')"
echo ""

echo "— Docker —————"
printf "  Containers: %s running\n" "$(docker ps -q | wc -l)"
printf "  Images:     %s\n" "$(docker images -q | wc -l)"
printf "  Disk usage: %s\n" "$(docker system df --format '{{.Size}}' | head
-1)"
echo ""
```

```

echo "— Containers —————"
docker ps --format "  {{.Names}}\t{{.Status}}" | column -t -s '$\t'
echo ""

echo "— Firewall —————"
ufw status | grep -E "^(Status|[0-9])" | head -5
echo ""

echo "— Ports —————"
ss -tlnp | grep -E ':(80|443|22)\s' | awk '{printf "  %s\n", $4}'
echo ""
echo "└───────────────────────────┘"
EOF

chmod +x ~/check-health.sh

```

Run it anytime:

```
~/check-health.sh
```

10. Quick Reference Card

VPS Setup — Complete Checklist

```

# — 1. FIRST LOGIN —————
ssh root@YOUR_DROPLET_IP

# — 2. UPDATE SYSTEM —————
apt update && apt upgrade -y
timedatectl set-timezone Asia/Kuala_Lumpur

# — 3. INSTALL ESSENTIALS —————
apt install -y curl wget git nano htop ncdu net-tools ca-certificates gnupg
lsb-release

# — 4. INSTALL DOCKER —————
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | tee
/etc/apt/sources.list.d/docker.list > /dev/null
apt update && apt install -y docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin
systemctl enable docker

```

```
# — 5. CONFIGURE FIREWALL —————
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp comment "SSH"
ufw allow 80/tcp comment "HTTP"
ufw allow 443/tcp comment "HTTPS"
ufw enable

# — 6. CREATE PROJECT DIRECTORY —————
mkdir -p ~/kanban-prod
cd ~/kanban-prod

# — 7. CREATE FILES (.env, Caddyfile, docker-compose.yml)
# (see sections 6.3, 6.4, 6.5 above)

# — 8. LOGIN TO GHCR —————
echo "ghp_YOUR_TOKEN" | docker login ghcr.io -u YOUR_USERNAME --password-stdin

# — 9. LAUNCH —————
docker compose up -d

# — 10. VERIFY —————
docker compose ps
docker compose logs --tail 20
curl -s http://localhost/api/health
```

Daily Operations

```
# SSH into server
ssh vps

# Check status
cd ~/kanban-prod && docker compose ps

# View logs
docker compose logs -f

# Manual update (if watchtower is slow)
docker compose pull && docker compose up -d

# Restart a service
docker compose restart api

# Check resource usage
docker stats --no-stream

# Check disk space
df -h / && docker system df
```

Emergency Commands

```
# Everything crashed – restart all
cd ~/kanban-prod
docker compose down
docker compose up -d

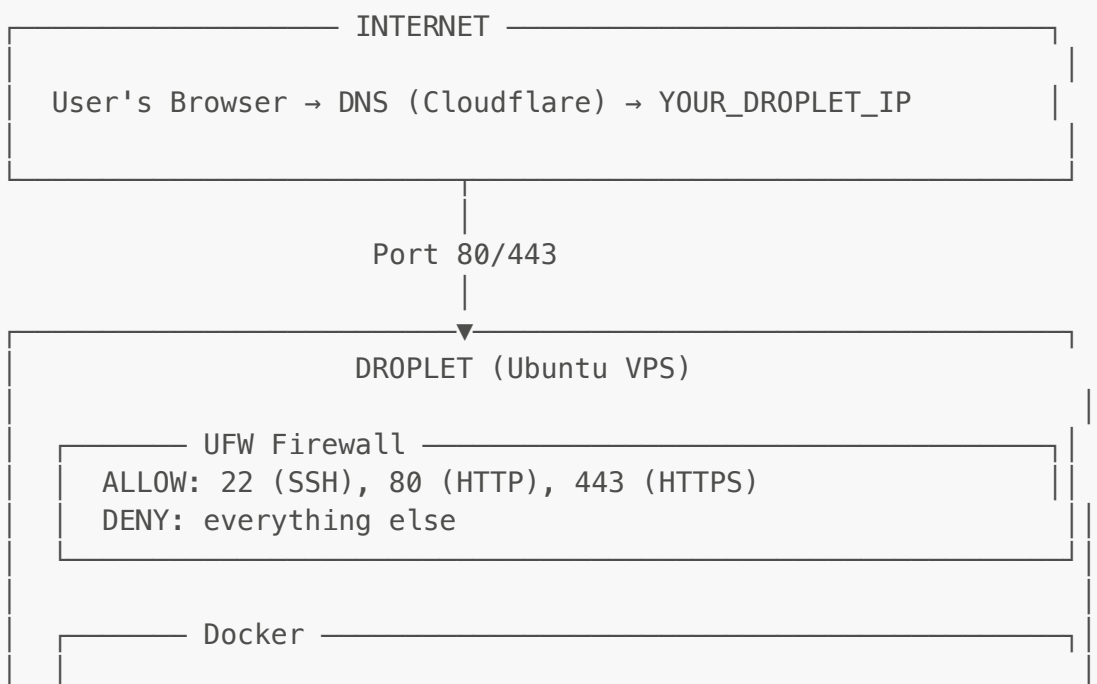
# Out of disk space
docker system prune -af
docker builder prune -af

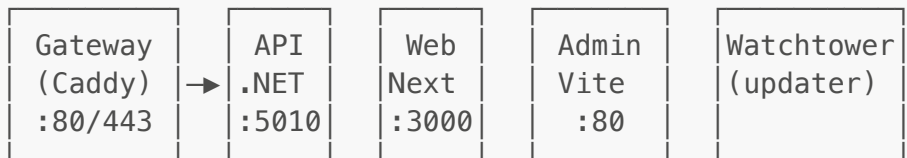
# Database connection failing
# Check if DB is reachable:
docker exec kanban-api sh -c "nc -zv DB_HOST 25060"

# SSL not working
# 1. Check DNS:
dig YOUR_SUBDOMAIN.lazuar.com
# 2. Check Caddy logs:
docker compose logs gateway | grep -i "error\|certificate"
# 3. Force SSL renewal:
docker compose restart gateway

# Completely start over
cd ~/kanban-prod
docker compose down -v
rm -rf ~/kanban-prod
mkdir ~/kanban-prod
# (recreate files and re-deploy)
```

Diagram: VPS Architecture





~/kanban-prod/
├ .env
├ Caddyfile
└ docker-compose.yml

Port 25060
↓

DIGITALOCEAN MANAGED POSTGRESQL
(Automatic backups, updates, SSL)